Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Kalavakkam – 603 110

Department of Computer Science & Engineering

Computer Networks Laboratory

1513 – Computer Networks Lab

Assignment 7

Simulation of Computer Networks using NS-3

Name:	Sundareswaran Rajaguru
Register Number:	3122247001048
Year / Semester:	III Year / V Semester
Course:	1513 Computer Networks Lab
Date of Submission:	26 August 2026

1. Objective

To install and use NS-3 (Network Simulator 3), a discrete-event C++ network simulator, to build and analyse two simple simulated networks:

- **Experiment 1:** a 4-node network with two simultaneous UDP traffic flows, measuring throughput and average end-to-end packet latency *without* using the `FlowMonitor` module.
- **Experiment 2:** a 2-node point-to-point link carrying two independent UDP applications distinguished only by destination port, measuring how throughput varies with link propagation delay.

2. Environment Setup

NS-3 was installed inside a fresh Ubuntu 24.04 environment (isolated in a Docker container so the host macOS system stays untouched) since NS-3 targets Linux/g++ toolchains.

```
apt-get install -y g++ python3 python3-dev cmake ninja-build git ccache
git clone --depth 1 --branch ns-3.42 https://gitlab.com/nsnam/ns-3-dev.git
cd ns-3-dev
./ns3 configure --build-profile=optimized --disable-examples --disable-tests \
    --enable-modules=core,network,internet,point-to-point,applications
./ns3 build -j4
```

Restricting `--enable-modules` to just the five modules actually used (`core`, `network`, `internet`, `point-to-point`, `applications`) and disabling examples/tests keeps the build small and fast without losing any functionality needed for either experiment.

3. Experiment 1 – Simple 4-Node Network

3.1 Topology

```
n0 ----- n1 ----- n2
          |
          n3
```

Three 5 Mbps / 2 ms point-to-point links connect the four nodes. Two UDP flows run concurrently: **Flow 1** (`n0`→`n2`, 1 Mbps) and **Flow 2** (`n3`→`n0`, 500 Kbps), for 10 simulated seconds.

3.2 Algorithm

1. Create four nodes and three point-to-point links (5 Mbps, 2 ms delay).
2. Install the Internet stack on all nodes and assign one `/24` subnet per link.
3. Populate global routing tables (`Ipv4GlobalRoutingHelper`) so multi-hop paths such as `n0`→`n1`→`n2` resolve automatically.
4. Install a `PacketSink` (receiver) on `n2` for Flow 1 and on `n0` for Flow 2; install an `OnOff` UDP source on `n0` (1 Mbps) and `n3` (500 Kbps) respectively, with `OnTime`/`OffTime` forced to always-on (the `OnOffHelper` default alternates 1 s on / 1 s off, which would otherwise halve the delivered rate below the flow's configured value).
5. Connect a callback to each source's `"Tx"` trace source (pushes the send timestamp onto a per-flow FIFO queue) and to each sink's `"Rx"` trace source (pops the oldest timestamp and accumulates `Now() - txTime` as the packet's latency). Because each flow travels a single ordered point-to-point path with no competing traffic on its own queue, packets always

arrive in send order, so FIFO matching gives the exact per-packet delay – `FlowMonitor` is never used.

6. Run the simulation for 10 seconds, then print total bytes received, throughput, and average latency per flow.

3.3 Code

```cpp
// Assignment 7 - Experiment 1: Simulation of a Simple Network using NS-3
// Topology:  n0 --- n1 --- n2
//                    |
//                    n3
// Flow 1: n0 -> n2 (UDP, 1 Mbps)   Flow 2: n3 -> n0 (UDP, 500 Kbps)
// Reports: total bytes received, throughput per flow, average latency per
// flow (computed with plain Tx/Rx trace callbacks -- no FlowMonitor used).

#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include <queue>

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("SimpleNetwork");

// FIFO queues of send timestamps; since each flow travels a single ordered
// point-to-point path with no other traffic sharing its queue slot order,
// packets arrive in the order they were sent, so matching front-of-queue
// Tx time with each Rx event gives the correct per-packet delay.
static std::queue<Time> txQ1, txQ2;
static double totalDelay1 = 0, totalDelay2 = 0;
static uint32_t rxCount1 = 0, rxCount2 = 0;

static void TxTrace1(Ptr<const Packet> p) { txQ1.push(Simulator::Now()); }
static void TxTrace2(Ptr<const Packet> p) { txQ2.push(Simulator::Now()); }

static void RxTrace1(Ptr<const Packet> p, const Address &a)
{
    if (!txQ1.empty()) {
        totalDelay1 += (Simulator::Now() - txQ1.front()).GetSeconds();
        txQ1.pop();
        rxCount1++;
    }
}
static void RxTrace2(Ptr<const Packet> p, const Address &a)
{
    if (!txQ2.empty()) {
        totalDelay2 += (Simulator::Now() - txQ2.front()).GetSeconds();
        txQ2.pop();
        rxCount2++;
    }
}

int main(int argc, char *argv[])
{
    CommandLine cmd;
    cmd.Parse(argc, argv);

    // 1. Create the four nodes
    NodeContainer nodes;
    nodes.Create(4);
```

```
56
57      // 2. Create the three point-to-point links (5 Mbps, 2 ms)
58      PointToPointHelper p2p;
59      p2p.SetDeviceAttribute("DataRate", StringValue("5Mbps"));
60      p2p.SetChannelAttribute("Delay", StringValue("2ms"));
61
62      NetDeviceContainer d01 = p2p.Install(nodes.Get(0), nodes.Get(1));
63      NetDeviceContainer d12 = p2p.Install(nodes.Get(1), nodes.Get(2));
64      NetDeviceContainer d32 = p2p.Install(nodes.Get(3), nodes.Get(2));
65
66      // 3. Internet stack on all nodes
67      InternetStackHelper internet;
68      internet.Install(nodes);
69
70      // 4. Assign IP addresses
71      Ipv4AddressHelper address;
72      address.SetBase("10.1.1.0", "255.255.255.0");
73      Ipv4InterfaceContainer i01 = address.Assign(d01);
74      address.SetBase("10.1.2.0", "255.255.255.0");
75      Ipv4InterfaceContainer i12 = address.Assign(d12);
76      address.SetBase("10.1.3.0", "255.255.255.0");
77      Ipv4InterfaceContainer i32 = address.Assign(d32);
78
79      // 5. Routing
80      Ipv4GlobalRoutingHelper::PopulateRoutingTables();
81
82      // 6. Flow 1: n0 -> n2, 1 Mbps
83      uint16_t port1 = 5000;
84      PacketSinkHelper sink1("ns3::UdpSocketFactory", InetSocketAddress(
            Ipv4Address::GetAny(), port1));
85      ApplicationContainer sinkApp1 = sink1.Install(nodes.Get(2));
86      sinkApp1.Start(Seconds(1.0));
87      sinkApp1.Stop(Seconds(10.0));
88
89      OnOffHelper source1("ns3::UdpSocketFactory", InetSocketAddress(i12.
            GetAddress(1), port1));
90      source1.SetAttribute("DataRate", StringValue("1Mbps"));
91      source1.SetAttribute("PacketSize", UintegerValue(512));
92      // Keep the source continuously "on" (default OnOffHelper alternates 1s
93      // on / 1s off, which would halve the effective rate below the 1 Mbps we
94      // actually want to demonstrate).
95      source1.SetAttribute("OnTime", StringValue("ns3::ConstantRandomVariable[
            Constant=100]"));
96      source1.SetAttribute("OffTime", StringValue("ns3::ConstantRandomVariable[
            Constant=0]"));
97      ApplicationContainer sourceApp1 = source1.Install(nodes.Get(0));
98      sourceApp1.Start(Seconds(2.0));
99      sourceApp1.Stop(Seconds(10.0));
100
101     // Flow 2: n3 -> n0, 500 Kbps
102     uint16_t port2 = 6000;
103     PacketSinkHelper sink2("ns3::UdpSocketFactory", InetSocketAddress(
            Ipv4Address::GetAny(), port2));
104     ApplicationContainer sinkApp2 = sink2.Install(nodes.Get(0));
105     sinkApp2.Start(Seconds(1.0));
106     sinkApp2.Stop(Seconds(10.0));
107
108     OnOffHelper source2("ns3::UdpSocketFactory", InetSocketAddress(i01.
            GetAddress(0), port2));
109     source2.SetAttribute("DataRate", StringValue("500Kbps"));
110     source2.SetAttribute("PacketSize", UintegerValue(512));
111     source2.SetAttribute("OnTime", StringValue("ns3::ConstantRandomVariable[
            Constant=100]"));
```

```
    source2.SetAttribute("OffTime", StringValue("ns3::ConstantRandomVariable[
        Constant=0]"));
    ApplicationContainer sourceApp2 = source2.Install(nodes.Get(3));
    sourceApp2.Start(Seconds(3.0));
    sourceApp2.Stop(Seconds(10.0));

    // 7. Hook Tx/Rx trace sources for latency measurement (no FlowMonitor)
    sourceApp1.Get(0)->TraceConnectWithoutContext("Tx", MakeCallback(&TxTrace1)
        );
    sinkApp1.Get(0)->TraceConnectWithoutContext("Rx", MakeCallback(&RxTrace1));
    sourceApp2.Get(0)->TraceConnectWithoutContext("Tx", MakeCallback(&TxTrace2)
        );
    sinkApp2.Get(0)->TraceConnectWithoutContext("Rx", MakeCallback(&RxTrace2));

    // 8. Run simulation for 10 seconds
    Simulator::Stop(Seconds(10.0));
    Simulator::Run();

    Ptr<PacketSink> sinkPtr1 = DynamicCast<PacketSink>(sinkApp1.Get(0));
    Ptr<PacketSink> sinkPtr2 = DynamicCast<PacketSink>(sinkApp2.Get(0));

    double activeTime1 = 10.0 - 2.0; // Flow 1 active window
    double activeTime2 = 10.0 - 3.0; // Flow 2 active window
    double thr1 = (sinkPtr1->GetTotalRx() * 8.0) / activeTime1 / 1e6; // Mbps
    double thr2 = (sinkPtr2->GetTotalRx() * 8.0) / activeTime2 / 1e6; // Mbps

    std::cout << "\n===== Flow 1 (n0 -> n2) =====\n";
    std::cout << "Total bytes received : " << sinkPtr1->GetTotalRx() << " bytes
        \n";
    std::cout << "Throughput           : " << thr1 << " Mbps\n";
    std::cout << "Avg packet latency    : "
              << (rxCount1 ? (totalDelay1 / rxCount1) * 1000 : 0) << " ms\n";

    std::cout << "\n===== Flow 2 (n3 -> n0) =====\n";
    std::cout << "Total bytes received : " << sinkPtr2->GetTotalRx() << " bytes
        \n";
    std::cout << "Throughput           : " << thr2 << " Mbps\n";
    std::cout << "Avg packet latency    : "
              << (rxCount2 ? (totalDelay2 / rxCount2) * 1000 : 0) << " ms\n";

    std::cout << "\n===== Comparison =====\n";
    std::cout << "Flow 1 offers " << (thr1 > thr2 ? "higher" : "lower")
              << " throughput than Flow 2 because its source rate (1 Mbps) is"
              << (thr1 > thr2 ? " higher" : " lower")
              << " than Flow 2's (500 Kbps); both stay well within the 5 Mbps"
              << " link capacity so latency for both is dominated by the fixed"
              << " 2 ms-per-hop propagation delay rather than queuing.\n";

    Simulator::Destroy();
    return 0;
}
```

3.4 Output / Results

```
===== Flow 1 (n0 -> n2) =====
Total bytes received : 998912 bytes
Throughput           : 0.998912 Mbps
Avg packet latency    : 5.7344 ms

===== Flow 2 (n3 -> n0) =====
Total bytes received : 436736 bytes
Throughput           : 0.499127 Mbps
```

```
Avg packet latency    : 8.6016 ms

===== Comparison =====
Flow 1 offers higher throughput than Flow 2 because its source rate (1 Mbps) is
     higher than Flow 2's (500 Kbps); both stay well within the 5 Mbps link
    capacity so latency for both is dominated by the fixed 2 ms-per-hop
    propagation delay rather than queuing.
```

3.5 Comparison of the Two Flows

Flow 1 (1 Mbps source) delivers roughly double the throughput of Flow 2 (500 Kbps source), exactly matching the ratio of their configured source rates, since neither flow comes close to saturating the 5 Mbps links (no queuing loss). Average latency is dominated by the fixed propagation delay of the path each flow takes (2 hops × 2 ms = 4 ms for Flow 1 via `n0`→`n1`→`n2`; 2 hops × 2 ms = 4 ms for Flow 2 via `n3`→`n2`→`n1`→`n0`, three hops) plus a small, roughly constant transmission/serialization delay per 512-byte packet – it does not depend meaningfully on the source data rate at these load levels.

4. Experiment 2 – UDP Traffic and Port-Based Application Delivery

4.1 Topology

```
Node1 ----------------- Node2
         P2P link
```

A single point-to-point link (10 Mbps, variable propagation delay) connects two nodes. Two independent UDP client/server application pairs run over it, one bound to port 5000 and the other to port 6000 – both use the same pair of IP addresses, so the destination port number is the only thing that lets Node2 route an incoming datagram to the correct server application.

4.2 Algorithm

1. Create two nodes and one point-to-point link whose propagation delay is given on the command line (`--delay`).
2. Install the Internet stack and assign one `/24` subnet to the link.
3. Install `UdpServer` on Node2 listening on port 5000, and `UdpClient` on Node1 sending to `Node2:5000` at a fixed 512-byte-packet rate (1 Mbps).
4. If `--twoFlows=true`, repeat the previous step with a second, independent `UdpServer`/`UdpClient` pair on port 6000.
5. Enable pcap tracing on both interfaces (`PointToPointHelper::EnablePcapAll`).
6. Run the simulation and print each server's received-packet count and the resulting throughput.

4.3 Code

```
1 // Assignment 7 - Experiment 2: UDP Traffic and Port-Based Application Delivery
2 // Topology: Node1 ---- P2P link ---- Node2
3 // Two independent UDP flows (port 5000 and port 6000) share the same pair of
4 // IP addresses; only the destination port tells the receiver which
5 // application a packet belongs to. Link latency and the number of active
6 // flows are passed on the command line so the same binary can reproduce
```

```cpp
// every row of both required result tables.

#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("UdpPortDelivery");

int main(int argc, char *argv[])
{
    std::string delay = "2ms";    // link propagation delay (varied per run)
    bool twoFlows = false;        // false = only Flow 1 (Sec.3), true = both (Sec.5/7)
    double simTime = 5.0;         // seconds each flow is active

    CommandLine cmd;
    cmd.AddValue("delay", "Point-to-point propagation delay", delay);
    cmd.AddValue("twoFlows", "Enable the second UDP flow (port 6000)", twoFlows);
    cmd.Parse(argc, argv);

    // 1. Two nodes, one point-to-point link
    NodeContainer nodes;
    nodes.Create(2);

    PointToPointHelper p2p;
    p2p.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
    p2p.SetChannelAttribute("Delay", StringValue(delay));
    NetDeviceContainer devices = p2p.Install(nodes);

    InternetStackHelper internet;
    internet.Install(nodes);

    Ipv4AddressHelper address;
    address.SetBase("10.1.1.0", "255.255.255.0");
    Ipv4InterfaceContainer interfaces = address.Assign(devices);
    Ipv4Address node2Addr = interfaces.GetAddress(1);

    // 2 & 3. Flow 1 (Application 1): UdpClient on Node1 -> UdpServer on Node2, port 5000
    uint16_t port1 = 5000;
    UdpServerHelper server1(port1);
    ApplicationContainer serverApp1 = server1.Install(nodes.Get(1));
    serverApp1.Start(Seconds(0.0));
    serverApp1.Stop(Seconds(simTime + 1.0));

    UdpClientHelper client1(node2Addr, port1);
    client1.SetAttribute("MaxPackets", UintegerValue(0xFFFFFFFF));
    client1.SetAttribute("PacketSize", UintegerValue(512));
    client1.SetAttribute("Interval", TimeValue(MicroSeconds(4096))); // 512B/4.096ms = 1 Mbps
    ApplicationContainer clientApp1 = client1.Install(nodes.Get(0));
    clientApp1.Start(Seconds(1.0));
    clientApp1.Stop(Seconds(simTime + 1.0));

    // 4. Flow 2 (Application 2): second UdpClient/UdpServer pair on port 6000
    uint16_t port2 = 6000;
    ApplicationContainer serverApp2, clientApp2;
    if (twoFlows) {
```

```
        UdpServerHelper server2(port2);
        serverApp2 = server2.Install(nodes.Get(1));
        serverApp2.Start(Seconds(0.0));
        serverApp2.Stop(Seconds(simTime + 1.0));

        UdpClientHelper client2(node2Addr, port2);
        client2.SetAttribute("MaxPackets", UintegerValue(0xFFFFFFFF));
        client2.SetAttribute("PacketSize", UintegerValue(512));
        client2.SetAttribute("Interval", TimeValue(MicroSeconds(4096)));
        clientApp2 = client2.Install(nodes.Get(0));
        clientApp2.Start(Seconds(1.0));
        clientApp2.Stop(Seconds(simTime + 1.0));
    }

    // 6. PCAP capture on both interfaces (view with Wireshark)
    p2p.EnablePcapAll("udp-port-based");

    Simulator::Stop(Seconds(simTime + 1.0));
    Simulator::Run();

    Ptr<UdpServer> srv1 = DynamicCast<UdpServer>(serverApp1.Get(0));
    double thr1 = (srv1->GetReceived() * 512.0 * 8.0) / simTime / 1e6;
    std::cout << "delay=" << delay << " Flow1(port5000) received=" << srv1->
        GetReceived()
              << " throughput=" << thr1 << "Mbps";

    if (twoFlows) {
        Ptr<UdpServer> srv2 = DynamicCast<UdpServer>(serverApp2.Get(0));
        double thr2 = (srv2->GetReceived() * 512.0 * 8.0) / simTime / 1e6;
        std::cout << " Flow2(port6000) received=" << srv2->GetReceived()
                  << " throughput=" << thr2 << "Mbps";
    }
    std::cout << std::endl;

    Simulator::Destroy();
    return 0;
}
```

4.4 Output / Results

```
===== Experiment 2 -- Single Flow (Port 5000 only), sweeping link latency =====
delay=2ms Flow1(port5000) received=1221 throughput=1.00024Mbps
delay=5ms Flow1(port5000) received=1220 throughput=0.999424Mbps
delay=10ms Flow1(port5000) received=1219 throughput=0.998605Mbps
delay=20ms Flow1(port5000) received=1216 throughput=0.996147Mbps

===== Experiment 2 -- Two Flows (Port 5000 + Port 6000), sweeping link latency
    =====
delay=2ms Flow1(port5000) received=1221 throughput=1.00024Mbps Flow2(port6000)
    received=1221 throughput=1.00024Mbps
delay=5ms Flow1(port5000) received=1220 throughput=0.999424Mbps Flow2(port6000)
     received=1220 throughput=0.999424Mbps
delay=10ms Flow1(port5000) received=1219 throughput=0.998605Mbps Flow2(port6000
    ) received=1219 throughput=0.998605Mbps
delay=20ms Flow1(port5000) received=1216 throughput=0.996147Mbps Flow2(port6000
    ) received=1216 throughput=0.996147Mbps

===== PCAP Verification (delay=2ms, both flows), filtered by destination port
    =====
-- Port 5000 (Flow 1 / Server 1), first 3 packets --
00:00:01.000000 IP 10.1.1.1.49153 > 10.1.1.2.5000: UDP, length 512
00:00:01.004096 IP 10.1.1.1.49153 > 10.1.1.2.5000: UDP, length 512
```

```
00:00:01.008192 IP 10.1.1.1.49153 > 10.1.1.2.5000: UDP, length 512
-- Port 6000 (Flow 2 / Server 2), first 3 packets --
00:00:01.000433 IP 10.1.1.1.49154 > 10.1.1.2.6000: UDP, length 512
00:00:01.004529 IP 10.1.1.1.49154 > 10.1.1.2.6000: UDP, length 512
00:00:01.008625 IP 10.1.1.1.49154 > 10.1.1.2.6000: UDP, length 512
-- Packet counts per port in the capture --
Port 5000 packets: 1221
Port 6000 packets: 1221
```

4.5 Result Tables

Link Latency	Throughput (Single Flow, Mbps)
2 ms	1.00024
5 ms	0.99942
10 ms	0.99861
20 ms	0.99615

Link Latency	Flow 1 Throughput (Mbps)	Flow 2 Throughput (Mbps)
2 ms	1.00024	1.00024
5 ms	0.99942	0.99942
10 ms	0.99861	0.99861
20 ms	0.99615	0.99615

4.6 Packet Capture Verification

With `EnablePcapAll("udp-port-based")` enabled, opening the generated `.pcap` files in Wireshark and filtering `udp.port==5000` / `udp.port==6000` confirms that every packet sent from Node1 to Node2 with destination port 5000 is captured arriving only at the trace point belonging to the Server-1 socket, and likewise port-6000 packets are the only ones delivered to the Server-2 socket – i.e. both flows share the identical IP source/destination pair, and only the transport-layer port number disambiguates which application instance a datagram belongs to. This mirrors how a real host multiplexes many simultaneous connections (e.g. a web server on port 80 and an SSH server on port 22) over one IP address.

4.7 Effect of Latency and a Second Flow

Because the link (10 Mbps) is never the bottleneck for either 1 Mbps UDP flow, throughput stays essentially constant as propagation delay increases from 2 ms to 20 ms – delay only shifts *when* each packet arrives, it does not reduce how many bits per second get through (UDP has no acknowledgement/congestion-control feedback loop for delay to interact with, unlike TCP). Adding the second flow on port 6000 does not reduce Flow 1's throughput either, for the same reason: 2 Mbps of combined UDP traffic is still far under the 10 Mbps link capacity, so both flows are delivered in full regardless of latency.

5. Learning Outcomes

- Understood NS-3's core abstractions – `Node`, `NetDevice`, `Channel`, `Application` – and how helper classes (`PointToPointHelper`, `InternetStackHelper`, `Ipv4AddressHelper`) compose them into a working simulated network in a few dozen lines of C++.

- Learned to measure per-packet latency using raw `"Tx"`/`"Rx"` trace-source callbacks instead of `FlowMonitor`, and understood why FIFO timestamp matching is valid on an uncontended point-to-point path.
- Understood how UDP port numbers, not IP addresses, are what let multiple independent applications on the same two hosts be multiplexed and demultiplexed correctly, and verified this at the packet level with Wireshark/pcap.
- Observed experimentally that propagation delay affects *latency* but not *throughput* for UDP flows that stay well under the link's bandwidth, since UDP has no rate-adaptive feedback loop.
- Practised parameterising an NS-3 simulation via `CommandLine` so a single compiled binary can reproduce an entire sweep of experimental conditions (multiple latencies, one vs. two flows) without recompiling.